

AWS Infrastructure Discovery via Terraform

Complete Technical Guide & Code Architecture Documentation

Executive Overview

This document details the complete end-to-end implementation for discovering existing AWS infrastructure using **Terraform Data Sources** instead of hardcoding resource identifiers or reading state files. By leveraging tags and dynamic queries, Terraform dynamically fetches the targeted VPC, Public Subnet, Security Group, and latest Amazon Linux 2023 AMI to provisions an EC2 instance in region `eu-west-1`.

Key Learning Objectives & Principles

1. Dynamic Infrastructure Discovery

Queries existing live resources using `tag:Name` filters. Eliminates brittle hardcoded IDs across environments.

2. AMI Auto-Resolution

Dynamically selects the latest Amazon Linux 2023 HVM x86_64 image matching versioning patterns directly from AWS catalog.

3. Strict Variable Separation

Defines explicit types and descriptions in `variables.tf` while keeping all values in `terraform.tfvars`.

4. Schema Compliance

Enforces correct schema separation—disallowing invalid arguments like `type` in output blocks.

1. Provider & Requirements Definition

FILE: VERSIONS.TF

```
terraform {
  required_version = ">= 1.5.7"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}
```

2. Input Variables Schema

FILE: VARIABLES.TF

```
variable "aws_region" {
  type        = string
  description = "The AWS region where resources are located."
}

variable "project_id" {
```

3. Lab Environment Variables

FILE: TERRAFORM.TFVARS

```
aws_region          = "eu-west-1"
project_id          = "cmtr-8k07hv2y"
vpc_name            = "cmtr-8k07hv2y-vpc"
public_subnet_name  = "cmtr-8k07hv2y-public-subnet-1"
security_group_name = "cmtr-8k07hv2y-sg"
```

4. Infrastructure Data Discovery

FILE: DATA.TF

```
# Discover existing VPC by its Name tag
data "aws_vpc" "selected" {
  filter {
    name   = "tag:Name"
    values = [var.vpc_name]
  }
}

# Discover existing Public Subnet by its Name tag and VPC ID
data "aws_subnet" "selected" {
  vpc_id = data.aws_vpc.selected.id

  filter {
    name   = "tag:Name"
    values = [var.public_subnet_name]
  }
}

# Discover existing Security Group by its Name tag and VPC ID
data "aws_security_group" "selected" {
  vpc_id = data.aws_vpc.selected.id

  filter {
    name   = "tag:Name"
    values = [var.security_group_name]
  }
}

# Discover latest Amazon Linux 2023 AMI
data "aws_ami" "al2023" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["al2023-ami-2023.*-x86_64"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }

  filter {
    name   = "architecture"
    values = ["x86_64"]
  }
}
```

6. Outputs Definition (Schema Corrected)

Note on Troubleshooting: Output blocks in Terraform do not support the `type` constraint. Types are automatically inferred from the `value` expression.

FILE: OUTPUTS.TF

```
output "discovered_vpc_id" {
  description = "The ID of the discovered VPC."
  value       = data.aws_vpc.selected.id
}

output "discovered_subnet_id" {
  description = "The ID of the discovered public subnet."
  value       = data.aws_subnet.selected.id
}

output "discovered_security_group_id" {
  description = "The ID of the discovered security group."
  value       = data.aws_security_group.selected.id
}

output "latest_ami_id" {
  description = "The ID of the discovered Amazon Linux 2023 AMI."
  value       = data.aws_ami.al2023.id
}

output "instance_id" {
  description = "The ID of the created EC2 instance."
  value       = aws_instance.cmtr-8k07hv2y-instance.id
}

output "instance_public_ip" {
  description = "The public IP address of the created EC2 instance."
  value       = aws_instance.cmtr-8k07hv2y-instance.public_ip
}
```

7. Execution Workflow & Verification Protocol

- Formatting & Initialization:** Run `terraform fmt` to align syntax according to HashiCorp standard, followed by `terraform init` to load the AWS provider plugin (v5.x).
- Validation & Dry Run:** Run `terraform validate` to ensure structural correctness. Run `terraform plan` to preview data source dynamic lookup results.
- Test Deployment:** Execute `terraform apply -auto-approve` to verify that the target VPC, subnet, security group, and AMI resolve properly and create the instance.
- Teardown (Crucial for Platform Verification):** Run `terraform destroy -auto-approve` to clean up created cloud resources prior to automated evaluation.
- Commit & Submit:** Push all local code files to your Git repository and provide the HTTPS URL with token to the automated evaluation platform.

Verification Success

All automated checks—file structure, provider initialization, resource tag-based data discovery, hardcoding omission, and EC2 binding—have successfully passed.